Reset Sequence Architecture

SONIC-MCU

System Reset Topology & Memory Retention | SPEC-RESET-v5.0

Rationale for Designated Reset Sequencing

The implementation of a managed reset sequence within the SONIC-MCU architecture is necessitated by both physical power dynamics and logical protocol constraints during chip initialization:

1. **Prevention of Shared Bus Contention:** The subsystem utilizes a multi-master AXI-Lite crossbar architecture containing both the RISC-V core and the Bootloader master interfaces. Waking up both master nodes simultaneously without sequencing would induce immediate contention. The uninitialized CPU could broadcast speculative memory requests while the Bootloader attempts to assert crossbar control for memory sideloading, leading to unpredictable bus deadlocks.
2. **Dynamic Glitch Mitigation and Power Optimization:** During early power ramp-up and initial memory population, peripheral inputs and floating control registers can generate undesirable clock glitches or meta-stable signal states. Enforcing an architectural sequence that maintains non-essential blocks under hard internal reset guarantees zero-power dynamic switching across unmapped logic matrices.
3. **Rapid System Restart and Debugging (Memory Retention):** During iterative software development or recovery from a watchdog timeout, re-fetching firmware from the external SPI flash introduces severe latency. A distinct Soft Reset allows the system to instantly zero-out processor and peripheral states while preserving the SRAM, enabling instantaneous re-execution of the loaded firmware.

Hardware Reset Topology

To eliminate the requirement for an active external system controller, the internal Bootloader engine is utilized as the Master Reset & Boot Controller. The architecture relies on four specific hardware reset signals to manage the system state safely.

System Reset Signals

- **Hard Reset (`hard_rst_ni`):** An external input pin utilized for full hardware initialization and power-up sequences.
- **Soft Reset (`soft_rst_ni`):** An external input pin utilized for system restarts that require memory retention.
- **Internal Peripheral Reset (`periph_rst_ni`):** An internally generated signal controlled by the Bootloader that drives the reset pins of the non-boot APB peripherals (I2C, Timer, UART, GPIO, CLIC).
- **Internal Core Reset (`core_rst_ni`):** An internally generated signal controlled by the Bootloader that exclusively drives the reset pin of the RISC-V CPU.

SRAM Data Protection Strategy

During a Soft Reset (`soft_rst_ni`), preserving SRAM integrity is critical. The Soft Reset signal is routed directly to the `axi_lite_sram_if` bridges. When asserted, it safely clears the bridge's internal AXI state machines while simultaneously clamping the physical SRAM Write-Enable (`sram_we_o`) to zero. This physically guarantees that the memory array cannot be corrupted during the reset transient.

Types of Boot

Cold Boot

A cold boot occurs when the system is powering on for the first time, meaning it needs to initialize everything from the ground up.

When the `hard_rst_ni` signal goes LOW, all peripherals, the AXI-Lite crossbar, and the RISC-V core are immediately reset, and the Bootloader FSM is forced into its `IDLE` phase.

When the `hard_rst_ni` signal goes HIGH again, the Bootloader uses the two internal resets (`core_rst_ni` and `periph_rst_ni`) to halt all other operations from starting. It then actively continues loading the instructions and data into the two SRAMs over the crossbar.

After the memory transfer is complete, the Bootloader concludes the transfer, parks its AXI Master channels, and drives both `core_rst_ni` and `periph_rst_ni` HIGH. The CPU awakens and fetches its first instruction from address `0x0`.

Warm Boot

A warm boot occurs when the system is "warm"—meaning the system already has power and is already working. During this process, most of the system is briefly placed into a dormant state without dropping the primary power rails.

When the `soft_rst_ni` signal is triggered, it resets all peripherals, the AXI-Lite crossbar, and the RISC-V core, but it explicitly leaves the instruction and data memory untouched.

Because the memory remains intact, this event bypasses the Bootloader's SPI fetching operations. The system simply clears the logic states and instantly releases the CPU, enabling an immediate execution restart from the existing firmware.